Cpt S 321 – Final Exam
Spring 2026
30 points total
Total pages: 6

First name:	
Last name:	
WSU ID:	

Read the instructions carefully until the end before asking questions:

- This is an individual exam: you are **not allowed to communicate solutions** with anyone.
- If something is unclear, ask for clarifications via the Canvas Discussion (no code allowed). If it is still unclear after my answer, then write down any assumptions that you are making.
- At the end, make sure you download your exam code in a clean directory to ensure that it works.
- No late submissions are allowed.
- **What to submit (failure to follow the submission instructions below will result in receiving 0 automatically for the exam):**

1. In your **in-class exercises** GitLab repository **create a new branch called** "FinalExam" and commit the following:
   - i) Design documents: a .PDF version of your class diagram and answers to the design table in Part 1. The design documents must be placed in a folder "Design_Documents" at the root of your project, and the name must contain your name and a description of the files (ex.: "**FirstName_LastName-final-ClassDiagram.pdf**" and "**FirstName_LastName-design-table-answers.pdf**").
   - ii) your code. Tag what you want to be graded with "FinalExam_DONE" tag.
2. Your GitLab readme file must explain what features you implemented for the exam and the ones that you are missing.
3. Create a pdf document with your AI statement for the HW (call it "AI_FinalExam"), stating whether or not you used GenAI and how and upload it in an "AI_Statements" folder. If you used GenAI for this Exam, 1) detail the process you followed in this document and 2) follow the guidelines for committing code generated by AI and adapted by you as outlined in the syllabus in the AI Statement section.
4. Record a video to demo your Exam (call it "Demo_FinalExam-FirstName_LastName").
5. On Canvas -> Assignments -> Upload your pre-recorded video by the deadline.
6. Sign up and demo with the TAs. (The sign-up sheet for the Final exam will be posted the week the exam is due.)

You are contacted by a company to build a **Subscription Platform Service** (i.e., design and implement a prototype as a **GUI Application in C#**) that allows users to subscribe to recurring product deliveries. Initially, the company will be starting with coffee as the only product. Customers will be able to customize their taste profiles and manage their subscriptions. The service partners with multiple grocery suppliers and offers a wide variety of coffees in different sizes and configurations.

The platform supports **multiple types of users**, each with different responsibilities and permissions. For all users, the behavior of the system must comply with the following:

- **The system must keep a log of all activities**, such as placing an order, modifying a subscription, updating taste preferences, rating coffees, etc.
- Each log entry includes the following information: **user, action, date and time, and description**.

As a start, the system must support the following user types:

- **Customer** – Customers can create and manage subscriptions, customize taste profiles, rate coffees, and view order history.
- **Supplier** – Suppliers manage coffee products and available inventory.
- **Subscription Manager** – Subscription managers configure subscription plans, delivery schedules, and pricing rules.
- **System Administrator** – Administrators can view logs and system-wide activity history

<u>Features for guests (no authentication required):</u>

1. **Register/create an account**: Users can create an account by providing their name, date of birth, email address, password, shipping address, and payment method.
2. **Login:** Using a username and a password, registered users should be able to log in to the system. Assume that you are using a third-party library for handling this, so your code should simply delegate to that library.
3. **Browse available coffees:** Allows users to browse coffees offered by different suppliers with the following information: Name, Origin, Roast type (light, medium, dark), Flavor notes (e.g., chocolate, fruity, nutty, caramel), Available bag sizes (e.g., 12 oz, 16 oz, 40 oz), Price per size.

<u>Features for customers (authentication required)</u>:

4. **Create and manage a taste profile:** Customers can create a taste profile that captures their coffee preferences, including preferred flavor notes (e.g., chocolate, fruity, nutty, floral), roast preferences, and strength preferences. The system will use this profile to recommend coffees for upcoming deliveries. Customers can update their taste profile at any time, and all changes must be logged.

5. **Create a subscription:** Customers can create a subscription by selecting: Coffee selection strategy (Manual selection, Recommendation-based selection using taste profile, or a mix of the two), Bag size per delivery, Delivery frequency (weekly, bi-weekly, monthly), Start date.
6. **Modify a subscription:** Customers can update an existing subscription by changing bag sizes, changing delivery frequency, changing coffee selection strategy, or pausing or resuming deliveries. All subscription changes must be logged.
7. **Generate weekly coffee recommendations:** The system provides personalized coffee recommendations based on the customer's taste profile and past ratings. Do not implement the recommendation algorithm. Instead, define the method signature and assume that it will be implemented by another team, but plan that different variations of this might need to be considered in the future.
8. **Place an order**: In addition to the subscription, the user should be able to place an order, providing the relevant information from the "**Create a subscription**" feature.
9. **Rate coffees**: After receiving a delivery, customers can rate coffees and provide feedback, which updates their taste profile over time.
10. **View activity history:** Customers can view their own activity logs, including profile updates, subscription changes, orders, and ratings.

Features for Suppliers (authentication required):

11. **Add a new coffee type:** Allows suppliers to add coffees with the following information: Name, Origin, Roast type (light, medium, dark), Flavor notes (e.g., chocolate, fruity, nutty, caramel), Available bag sizes (e.g., 12 oz, 16 oz, 40 oz), Price per size

Features for Subscription Manager (authentication required):

12. **Adapt pricing rules:** Allows subscription managers to provide discounts based on different rules. For example, customer loyalty, temporary discounts, or others.

Features for System Administrators (authentication required):

13. **View activity history**: System administrators can view logs for all users.

The application will contain multiple parts (views) in the GUI, and all of them will need to be updated as the state of the application changes. Those views could be grouping features specific to the users but other grouping are welcome.

Part 1. Design (16 points).

Draw a class diagram for your design to show how different classes are connected. Also highlight any additional design choices that you have made that were not discussed in the table by annotating your diagram with comments. Use draw.io to make the class diagram, export it as a **.PDF file,** and include it in your submission.

In addition to the class diagram, fill out the table below, indicating the **principles, patterns, and good practices that we learned about in class** that you plan to use for your design. Add more rows if needed.

For full points on this part, the design decisions, principles, and patterns should be properly implemented in the code.

Principle/Pattern name (type) Ex.: "Polymorphism (GRASP)"	**Classes that are involved** Ex.: "Animal, Cat, Dog"	**Additional comment** Ex.: Cat and Dog inherit from Animal

Part 2. Implementation (14 points).

Implement a prototype application using C# and a GUI of your choice (WinForms or Avalonia) and best coding practice.

Your solution should implement **8 features from the 13 listed in the problem description, with at least 1 feature from each category of users**.

Homework Grading Rubric and Points Breakdown	
CR1: Functionality (8 points)	● Your software fulfills the requirements of the features you have selected to implement with no inaccuracies in the output and no crashes. 1 point for each feature.
CR2 Quality of Version Control (1 point)	● Homework should be built iteratively (i.e., one feature at a time, not in one huge commit). ● Each commit should have cohesive functionality. ● Commit messages should concisely describe what is being committed. ● TDD should be used (i.e, write and commit tests first and then implement and commit functionality). ● Include "TDD" in all commit messages with tests that are written before the functionality is implemented. ● Use of a .gitignore. ● Commenting is done as the homework is built (i.e, there is commenting added in each commit, not done all at once at the end).
CR3: Quality of Identifiers (1 point)	● No underscores in names of classes, attributes, and properties. ● No numbers in names of classes or tests. ● Identifiers should be descriptive. ● Project names should make sense. ● Class names and method names use PascalCasing. ● Method arguments and local variables use camelCasing. ● No Linguistic Antipatterns or Lexicon Bad Smells.
CR4: Existence and Quality of Comments (1 point)	● Every method, attribute, type, and test case has a leading comment block with a minimum of <summary>, <returns>, <param>, and <exception> filled in as applicable. ● All comment blocks use the format that is generated when typing "///" on the line above each entity. ● There is useful inline commenting in addition to leading

	comment blocks that explains how the algorithm is implemented as needed.
CR5: Quality of Code (1 point)	● Each file should only contain one public class. ● Correct use of access modifiers. ● Classes are cohesive. ● Namespaces make sense. ● Code is easy to follow. ● StyleCop is installed and configured correctly for all projects in the solution and all warnings are resolved. If any warnings are suppressed, a good reason must be provided. ● Use of appropriate design patterns and software principles seen in class.
CR6: Existence and Quality of Tests (1 point)	● Normal, boundary, and overflow/error cases should be tested – tests should be written with NUnit. ● Test should be modularized (i.e, have a separate test case for each functionality - do not combine them into one large test case). ● *Note: In assignments with a GUI, we do not require testing of the GUI itself.*
CR7: Demo video – 5 min max (1 point)	● Check the "Pre-recorded demos instructions" in the additional resources for details on the items below. ● Version control history overview (< 30 sec.) ● AI statement (< 30 sec.) ● Design of the solution (2 min max) ● Features showcase (2 min max)